ECE590 Homework 3

Zanwen Fu (zf93)

Mar 28 2026

1 Question 1: Logistic Regression

1.1 (a) Model Setup and Data Description

We address the binary classification task of distinguishing digit "4" from digit "9" in the MNIST dataset using logistic regression.

Input representation. Each grayscale image in MNIST has dimensions 28×28 pixels. We flatten each image into a vector $x_i \in \mathbb{R}^{784}$ and normalize pixel values to $[0, 1]$ by dividing by 255.

Model. The logistic regression model parameterizes the conditional probability of class $Y = 4$ as:

$$p(Y = 4 \mid X = x_i) = \sigma(w^\top x_i + b), \tag{1}$$

where $w \in \mathbb{R}^{784}$ is the weight vector, $b \in \mathbb{R}$ is the bias, and $\sigma(\alpha) = \frac{e^\alpha}{1+e^\alpha}$ is the sigmoid function. Consequently, $p(Y = 9 \mid X = x_i) = 1 - \sigma(w^\top x_i + b)$. The model has a total of $784 + 1 = 785$ trainable parameters.

Loss function. We minimize the binary cross-entropy (BCE) loss over the training set:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \left[y_i \log \sigma(w^\top x_i + b) + (1 - y_i) \log \Big(1 - \sigma(w^\top x_i + b)\Big) \right], \tag{2}$$

where $y_i = 1$ if the true digit is 4 and $y_i = 0$ if it is 9. In practice, we use PyTorch's `BCEWithLogitsLoss`, which fuses the sigmoid and BCE computation for numerical stability.

Dataset statistics. We filter the standard MNIST training and test sets to retain only images labeled 4 or 9. The original MNIST training split (11,791 samples after filtering) is further divided into a 90% training set and a 10% validation set (used for model selection). Table 1 summarizes the data splits.

Table 1: Dataset statistics after filtering for digits 4 and 9.

Split	**Total Samples**	**Digit 4**	**Digit 9**
Training	10,612	—	—
Validation	1,179	—	—
Train + Val	11,791	5,842	5,949
Test	1,991	982	1,009

The dataset is approximately balanced between the two classes in both training and test sets (roughly 49.5% fours and 50.5% nines).

1.2 (b) Results

Hyperparameters. The model was trained using Stochastic Gradient Descent (SGD) with the hyperparameters listed in Table 2. We select the model checkpoint with the highest validation accuracy across all 50 epochs.

Table 2: Training hyperparameters.

Hyperparameter	**Value**
Optimizer	SGD
Learning rate	0.01
Batch size	64
Number of epochs	50
Model selection	Best validation accuracy

Test accuracy. The logistic regression model achieves a **test accuracy of 96.48%** (1,921 correct out of 1,991 test samples). The best validation accuracy during training was 95.84%.

Training curves. Figure 1 shows that both training and validation losses decrease steadily over 50 epochs, with no sign of significant overfitting (the two curves remain close). The validation accuracy rises quickly in the first 10 epochs and plateaus around 95.8%.

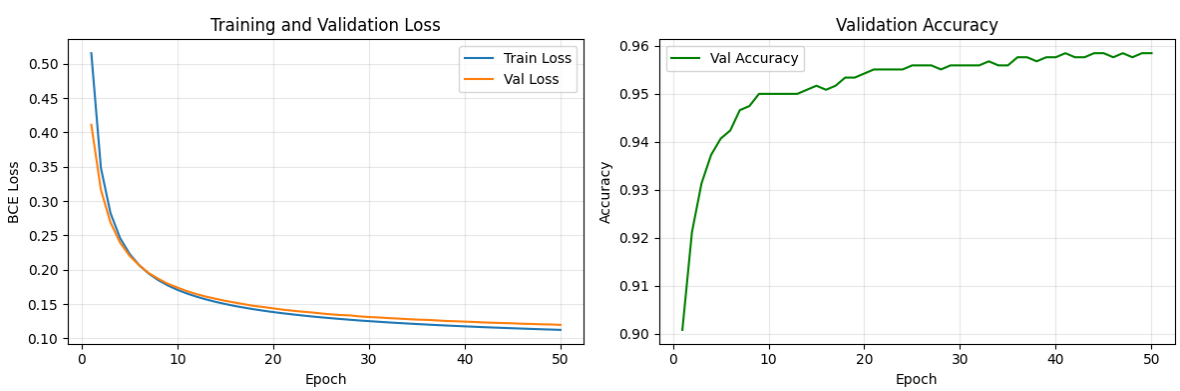

Figure 1: Training and validation loss (left) and validation accuracy (right) over 50 epochs of SGD.

Learned weight vector. Figure 2 visualizes the learned weight vector $w \in \mathbb{R}^{784}$ reshaped into a 28×28 image. Red (positive) pixel weights increase the predicted probability $p(Y = 4 \mid X)$, while blue (negative) weights increase the probability of class 9. The learned bias is $b = 0.1377$.

The weight image reveals an interpretable pattern: strong positive weights (red) appear in regions characteristic of the digit 4, such as the vertical stroke on the right side and the horizontal crossbar in the middle. Strong negative weights (blue) appear in the upper-center region where the loop of digit 9 typically occurs. This demonstrates that the linear model has learned to discriminate between the two digits by identifying the key structural differences in their shapes.

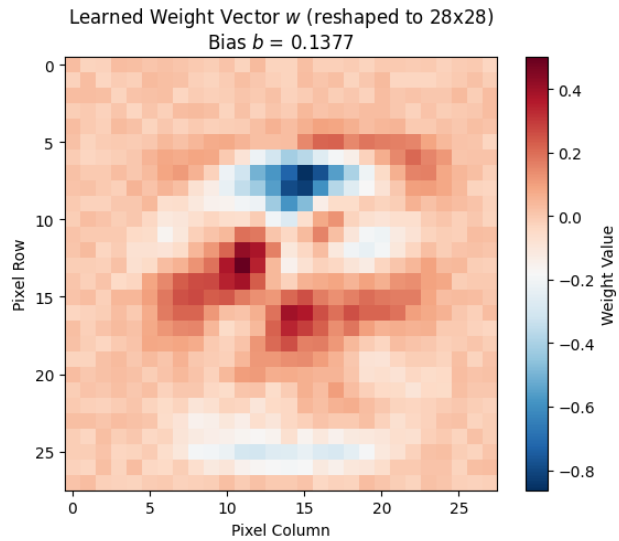

Figure 2: Learned weight vector w reshaped to 28×28. Red pixels increase $p(Y=4)$; blue pixels increase $p(Y=9)$. Learned bias $b = 0.1377$.

1.3 (c) Python Code

The complete implementation is shown below. The code was executed on Google Colab with GPU acceleration (PyTorch 2.10.0, CUDA enabled). The full notebook with running results is also available as `Q1_Logistic_Regression.ipynb`.

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np
```

Listing 1: Imports.

```python
transform = transforms.Compose([transforms.ToTensor()])
train_dataset_full = datasets.MNIST(root='./data', train=True,
                                    download=True, transform=transform)
test_dataset_full = datasets.MNIST(root='./data', train=False,
                                   download=True, transform=transform)

def filter_4_and_9(dataset):
    """Extract only digits 4 and 9, relabel as binary (4->1, 9->0)."""
    mask = (dataset.targets == 4) | (dataset.targets == 9)
    images = dataset.data[mask].float() / 255.0
    labels = dataset.targets[mask]
    binary_labels = (labels == 4).float()
    images_flat = images.view(-1, 28 * 28)
    return images_flat, binary_labels

X_train_full, y_train_full = filter_4_and_9(train_dataset_full)
X_test, y_test = filter_4_and_9(test_dataset_full)
```

Listing 2: Data loading and preprocessing.

```python
n_total = len(X_train_full)
n_val = int(0.1 * n_total)
n_train = n_total - n_val

torch.manual_seed(42)
indices = torch.randperm(n_total)
train_indices = indices[:n_train]
val_indices = indices[n_train:]

X_train, y_train = X_train_full[train_indices], y_train_full[train_indices]
X_val, y_val = X_train_full[val_indices], y_train_full[val_indices]

batch_size = 64
train_loader = DataLoader(TensorDataset(X_train, y_train),
                          batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val),
                        batch_size=batch_size, shuffle=False)
test_loader = DataLoader(TensorDataset(X_test, y_test),
                         batch_size=batch_size, shuffle=False)
```

Listing 3: Train/validation split and DataLoaders.

```python
class LogisticRegression(nn.Module):
    def __init__(self, input_dim):
        super(LogisticRegression, self).__init__()
        self.linear = nn.Linear(input_dim, 1)

    def forward(self, x):
        return self.linear(x)  # Returns logits

model = LogisticRegression(input_dim=784)
```

Listing 4: Model definition.

```python
criterion = nn.BCEWithLogitsLoss()
learning_rate = 0.01
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
num_epochs = 50

train_losses, val_losses, val_accuracies = [], [], []
best_val_acc = 0.0
best_model_state = None

for epoch in range(num_epochs):
    # Training
    model.train()
    epoch_loss = 0.0
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        logits = model(X_batch).squeeze()
        loss = criterion(logits, y_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(X_batch)
    train_losses.append(epoch_loss / len(X_train))

    # Validation
    model.eval()
```

```python
    val_loss, correct = 0.0, 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            logits = model(X_batch).squeeze()
            loss = criterion(logits, y_batch)
            val_loss += loss.item() * len(X_batch)
            preds = (torch.sigmoid(logits) >= 0.5).float()
            correct += (preds == y_batch).sum().item()
    val_losses.append(val_loss / len(X_val))
    val_accuracies.append(correct / len(X_val))

    if val_accuracies[-1] > best_val_acc:
        best_val_acc = val_accuracies[-1]
        best_model_state = {k: v.clone()
                            for k, v in model.state_dict().items()}

model.load_state_dict(best_model_state)
```

Listing 5: Training loop with SGD.

```python
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        logits = model(X_batch).squeeze()
        preds = (torch.sigmoid(logits) >= 0.5).float()
        correct += (preds == y_batch).sum().item()
        total += len(y_batch)
test_accuracy = correct / total
print(f"TEST ACCURACY: {test_accuracy:.4f} ({correct}/{total})")
```

Listing 6: Test evaluation.

```python
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ax1.plot(range(1, num_epochs+1), train_losses, label='Train Loss')
ax1.plot(range(1, num_epochs+1), val_losses, label='Val Loss')
ax1.set_xlabel('Epoch'); ax1.set_ylabel('BCE Loss')
ax1.set_title('Training and Validation Loss')
ax1.legend(); ax1.grid(True, alpha=0.3)

ax2.plot(range(1, num_epochs+1), val_accuracies,
         label='Val Accuracy', color='green')
ax2.set_xlabel('Epoch'); ax2.set_ylabel('Accuracy')
ax2.set_title('Validation Accuracy')
ax2.legend(); ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('q1_training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
```

Listing 7: Training curves visualization.

```python
w = model.linear.weight.data.squeeze().numpy().reshape(28, 28)
b = model.linear.bias.data.item()

fig, ax = plt.subplots(1, 1, figsize=(6, 5))
im = ax.imshow(w, cmap='RdBu_r', interpolation='nearest')
ax.set_title(f'Learned Weight Vector w (28x28)\nBias b = {b:.4f}')
ax.set_xlabel('Pixel Column'); ax.set_ylabel('Pixel Row')
```

```
plt.colorbar(im, ax=ax, label='Weight Value')
plt.tight_layout()
plt.savefig('q1_learned_weights.png', dpi=150, bbox_inches='tight')
plt.show()
```

Listing 8: Learned weight vector visualization.

2 Question 2: Multilayer Perceptron

2.1 (a) Model Setup

We extend the logistic regression model from Question 1 to a multilayer perceptron (MLP) of the form specified in the problem:

$$p(Y = 4 \mid X = x_i) = \sigma(w^\top z_i + b), \tag{3}$$

where

$$z_i = B \cdot f(Ax_i + \beta), \tag{4}$$

with $x_i \in \mathbb{R}^d$, $A \in \mathbb{R}^{d_1 \times d}$, $\beta \in \mathbb{R}^{d_1}$, $B \in \mathbb{R}^{d_2 \times d_1}$, $w \in \mathbb{R}^{d_2}$, and $b \in \mathbb{R}$.

Architecture choices. We set $d = 784$ (flattened input), $d_1 = 128$, and $d_2 = 64$. The nonlinearity $f(\cdot) = \mathrm{ReLU}(\cdot) = \max(0, \cdot)$ is applied element-wise to the vector $Ax_i + \beta$. The full forward pass is:

$$x_i \in \mathbb{R}^{784} \xrightarrow{A,\beta} \mathbb{R}^{128} \xrightarrow{\mathrm{ReLU}} \mathbb{R}^{128} \xrightarrow{B} \mathbb{R}^{64} \xrightarrow{w,b} \mathbb{R}^{1}.$$

Design rationale.

- $d_1 = 128$**:** The first hidden layer maps from 784 dimensions down to 128, providing a compressed representation that captures the most informative features of the input. This is large enough to learn a rich set of local feature detectors while remaining computationally efficient.
- $d_2 = 64$**:** The second layer further compresses from 128 to 64 dimensions, forming a more abstract representation before the final classification.
- **ReLU:** We chose $f(\cdot) = \mathrm{ReLU}$ as the nonlinearity because it avoids vanishing gradient issues, promotes sparse activations, and is the most widely used activation function for hidden layers.
- B **has no bias term:** The matrix B is implemented without a bias vector, matching the problem formulation $z_i = B \cdot f(Ax_i + \beta)$ exactly.

Parameter count. Table 3 shows the breakdown of all trainable parameters.

Table 3: MLP parameter breakdown.

Parameter	**Shape**	**Count**
A (first-layer weights)	128×784	100,352
β (first-layer bias)	128	128
B (second-layer weights)	64×128	8,192
w (output weights)	1×64	64
b (output bias)	1	1
Total		**108,737**

Loss function. Same as Question 1: binary cross-entropy (Eq. 2), implemented via `BCEWithLogitsLoss`.

Training details. The model was trained using SGD with the hyperparameters in Table 4. We employ early stopping with a patience of 10 epochs: if the validation accuracy does not improve for 10 consecutive epochs, training is halted and the model with the best validation accuracy is restored.

Table 4: MLP training hyperparameters.

Hyperparameter	**Value**
Optimizer	SGD
Learning rate	0.01
Batch size	64
Max epochs	100
Early stopping	Patience = 10 (on validation accuracy)

The data split is identical to Question 1: 10,612 training samples, 1,179 validation samples, and 1,991 test samples.

2.2 (b) Test Performance

The MLP achieves a **test accuracy of 97.84%** (1,948 correct out of 1,991 test samples). This is a meaningful improvement over the logistic regression model from Question 1 (96.48%), demonstrating the benefit of the nonlinear hidden layer representation.

Training was stopped early at epoch 64 (out of a maximum of 100) due to the patience criterion. The best validation accuracy was 98.05%.

Training curves. Figure 3 shows the training and validation loss curves and the validation accuracy over epochs. Both losses decrease steadily, with the training loss falling slightly below the validation loss in later epochs, indicating mild overfitting that the early stopping mechanism helps mitigate.

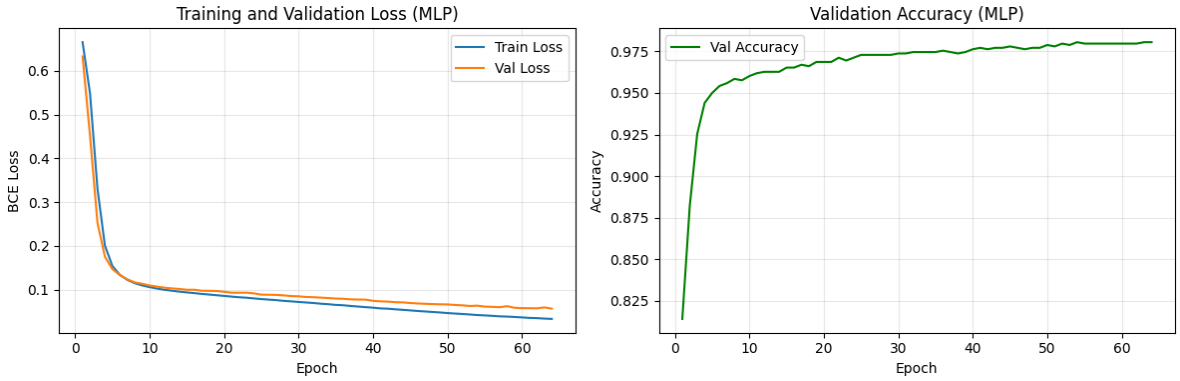

Figure 3: Training and validation loss (left) and validation accuracy (right) for the MLP over 64 epochs (early stopping triggered at epoch 64).

Comparison with logistic regression. Table 5 compares the two models.

Table 5: Comparison of logistic regression (Q1) and MLP (Q2).

Model	**Parameters**	**Val Accuracy**	**Test Accuracy**
Logistic Regression (Q1)	785	95.84%	96.48%
MLP (Q2)	108,737	98.05%	97.84%

The MLP's additional capacity (138× more parameters) and nonlinear hidden representations yield a 1.36 percentage point improvement in test accuracy.

First-layer weight visualization. Figure 4 shows the first 32 rows of the matrix $A \in \mathbb{R}^{128\times 784}$, each reshaped to 28×28. Unlike the single interpretable weight image in logistic regression (Q1), the MLP learns a distributed set of local feature detectors. Each hidden unit responds to a different spatial pattern, and the combination of these features through layers B and w produces the final classification.

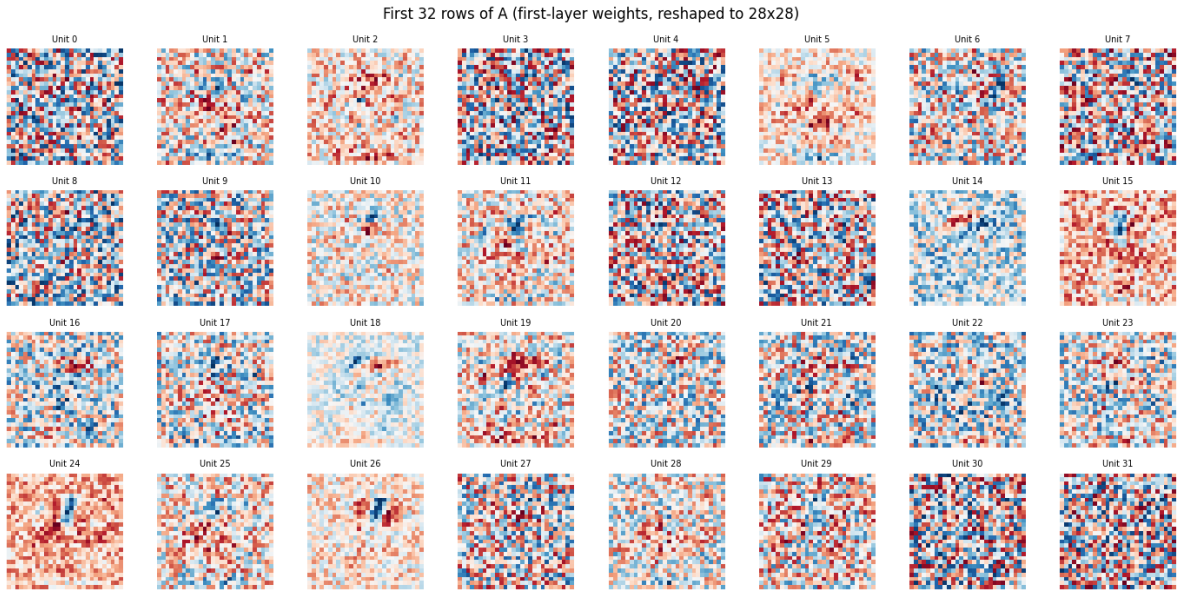

Figure 4: First 32 rows of the weight matrix A (out of 128), each reshaped to 28×28. Each image represents the input pattern that a single hidden unit responds to.

2.3 (c) Python Code

The complete implementation is shown below. The code was executed on Google Colab with GPU acceleration (PyTorch 2.10.0, CUDA enabled).

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Listing 9: Imports and device setup.

```
transform = transforms.Compose([transforms.ToTensor()])
train_dataset_full = datasets.MNIST(root='./data', train=True,
                                    download=True, transform=transform)
test_dataset_full = datasets.MNIST(root='./data', train=False,
                                   download=True, transform=transform)

def filter_4_and_9(dataset):
    mask = (dataset.targets == 4) | (dataset.targets == 9)
    images = dataset.data[mask].float() / 255.0
    labels = dataset.targets[mask]
    binary_labels = (labels == 4).float()
    images_flat = images.view(-1, 28 * 28)
    return images_flat, binary_labels

X_train_full, y_train_full = filter_4_and_9(train_dataset_full)
X_test, y_test = filter_4_and_9(test_dataset_full)

n_total = len(X_train_full)
n_val = int(0.1 * n_total)
torch.manual_seed(42)
indices = torch.randperm(n_total)
X_train = X_train_full[indices[:n_total - n_val]]
y_train = y_train_full[indices[:n_total - n_val]]
X_val = X_train_full[indices[n_total - n_val:]]
y_val = y_train_full[indices[n_total - n_val:]]

batch_size = 64
train_loader = DataLoader(TensorDataset(X_train, y_train),
                          batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val),
                        batch_size=batch_size, shuffle=False)
test_loader = DataLoader(TensorDataset(X_test, y_test),
                         batch_size=batch_size, shuffle=False)
```

Listing 10: Data loading (identical to Q1).

```
class MLP(nn.Module):
    """
    z_i = B * f(A * x_i + beta)
    p(Y=4 | X=x_i) = sigma(w^T z_i + b)
    """
    def __init__(self, input_dim=784, d1=128, d2=64):
        super(MLP, self).__init__()
        self.layer_A = nn.Linear(input_dim, d1)     # A, beta
        self.f = nn.ReLU()                           # f(.)
        self.layer_B = nn.Linear(d1, d2, bias=False) # B (no bias)
        self.output = nn.Linear(d2, 1)               # w, b

    def forward(self, x):
        hidden = self.f(self.layer_A(x))
        z = self.layer_B(hidden)
        logit = self.output(z)
        return logit

model = MLP(input_dim=784, d1=128, d2=64).to(device)
```

Listing 11: MLP model definition.

```
criterion = nn.BCEWithLogitsLoss()
learning_rate = 0.01
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
num_epochs = 100
patience = 10

train_losses, val_losses, val_accuracies = [], [], []
best_val_acc = 0.0
best_model_state = None
epochs_no_improve = 0

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        logits = model(X_batch).squeeze()
        loss = criterion(logits, y_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(X_batch)
    train_losses.append(epoch_loss / len(X_train))

    model.eval()
    val_loss, correct = 0.0, 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            logits = model(X_batch).squeeze()
            loss = criterion(logits, y_batch)
            val_loss += loss.item() * len(X_batch)
            preds = (torch.sigmoid(logits) >= 0.5).float()
            correct += (preds == y_batch).sum().item()
    val_losses.append(val_loss / len(X_val))
    val_accuracies.append(correct / len(X_val))

    if val_accuracies[-1] > best_val_acc:
        best_val_acc = val_accuracies[-1]
        best_model_state = {k: v.clone()
                            for k, v in model.state_dict().items()}
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1

    if epochs_no_improve >= patience:
        break

model.load_state_dict(best_model_state)
```

Listing 12: Training loop with early stopping.

```
model.eval()
correct, total = 0, 0
with torch.no_grad():
```

```
    for X_batch, y_batch in test_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        logits = model(X_batch).squeeze()
        preds = (torch.sigmoid(logits) >= 0.5).float()
        correct += (preds == y_batch).sum().item()
        total += len(y_batch)
test_accuracy = correct / total
print(f"TEST ACCURACY: {test_accuracy:.4f} ({correct}/{total})")
```

Listing 13: Test evaluation.

```
# Training curves
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
actual_epochs = len(train_losses)
ax1.plot(range(1, actual_epochs+1), train_losses, label='Train Loss')
ax1.plot(range(1, actual_epochs+1), val_losses, label='Val Loss')
ax1.set_xlabel('Epoch'); ax1.set_ylabel('BCE Loss')
ax1.set_title('Training and Validation Loss (MLP)')
ax1.legend(); ax1.grid(True, alpha=0.3)
ax2.plot(range(1, actual_epochs+1), val_accuracies,
         label='Val Accuracy', color='green')
ax2.set_xlabel('Epoch'); ax2.set_ylabel('Accuracy')
ax2.set_title('Validation Accuracy (MLP)')
ax2.legend(); ax2.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('q2_training_curves.png', dpi=150, bbox_inches='tight')
plt.show()
```

Listing 14: Training curves and weight visualization.

3 Question 3: Ten-Way MNIST Classification

We extend the MLP from Question 2 to the full 10-class MNIST classification problem (digits 0–9). The model now uses a softmax output layer:

$$p(Y = y \mid X = x_i) = \frac{\exp(w_y^\top z_i + b_y)}{\sum_{c=0}^{9} \exp(w_c^\top z_i + b_c)}, \tag{5}$$

where $z_i = B \cdot f(Ax_i + \beta)$ is the same hidden representation as in Question 2, and $y \in \{0, 1, \ldots, 9\}$.

Identifiability constraint. As specified, we fix $w_9 = \mathbf{0} \in \mathbb{R}^{d_2}$ and $b_9 = 0$ as the reference class. This is implemented by having the output layer produce only 9 logits (for classes 0–8) and concatenating a zero logit for class 9 before the softmax. This ensures the model is identifiable, since the softmax is invariant to adding a constant vector to all logits.

Architecture. The hidden layer structure is identical to Question 2:

$$x_i \in \mathbb{R}^{784} \xrightarrow{A,\beta} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{128} \xrightarrow{B} \mathbb{R}^{64} \xrightarrow{W,b} \mathbb{R}^{10} \xrightarrow{\text{softmax}} [0,1]^{10},$$

with $d_1 = 128$, $d_2 = 64$, and $f(\cdot) = \text{ReLU}$. The output layer learns 9 weight vectors $w_0, \ldots, w_8 \in \mathbb{R}^{64}$ and 9 biases $b_0, \ldots, b_8 \in \mathbb{R}$, while w_9 and b_9 are fixed at zero.

Parameter count. Table 6 shows the parameter breakdown. Compared to Q2, the output layer grows from $(64 + 1) = 65$ parameters to $(9 \times 64 + 9) = 585$ parameters, a modest increase.

Table 6: Q3 MLP parameter breakdown.

Parameter	**Shape**	**Count**
A (first-layer weights)	128×784	100,352
β (first-layer bias)	128	128
B (second-layer weights)	64×128	8,192
W_{out} (output weights, classes 0–8)	9×64	576
b_{out} (output biases, classes 0–8)	9	9
Total		**109,257**

Loss function. We minimize the cross-entropy loss:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^{N} \log p(Y = y_i \mid X = x_i), \tag{6}$$

implemented via PyTorch's `CrossEntropyLoss`, which combines the softmax and negative log-likelihood in a numerically stable fashion.

Dataset. We use the full MNIST dataset: 60,000 training images and 10,000 test images across all 10 digit classes. The training set is split 90/10 into 54,000 training and 6,000 validation samples.

Training details. The training setup follows Table 7. Early stopping with patience 10 halted training at epoch 67.

Table 7: Q3 training hyperparameters.

Hyperparameter	**Value**
Optimizer	SGD
Learning rate	0.01
Batch size	64
Max epochs	100
Early stopping	Patience = 10 (on validation accuracy)
Loss function	Cross-entropy (softmax + NLL)

Test performance. The 10-class MLP achieves a **test accuracy of 97.43%** (9,743 correct out of 10,000 test samples). The best validation accuracy during training was 97.03%.

Per-class accuracy. Table 8 reports the per-class accuracy on the test set. All classes achieve above 95% accuracy, with digits 0 and 1 being the easiest to classify (98.78% and 98.77%) and digit 9 being the hardest (95.24%). The slightly lower accuracy for class 9 may be partly attributed to its role as the reference class with fixed zero parameters.

Table 8: Per-class test accuracy for the 10-class MLP.

Digit	**Correct**	**Total**	**Accuracy**
0	968	980	98.78%
1	1121	1135	98.77%
2	1007	1032	97.58%
3	989	1010	97.92%
4	954	982	97.15%
5	866	892	97.09%
6	937	958	97.81%
7	998	1028	97.08%
8	942	974	96.71%
9	961	1009	95.24%

Confusion matrix. Figure 5 shows the confusion matrix on the test set. The strong diagonal confirms that the model performs well across all classes. The most common confusions involve visually similar digit pairs such as 4↔9 and 3↔5.

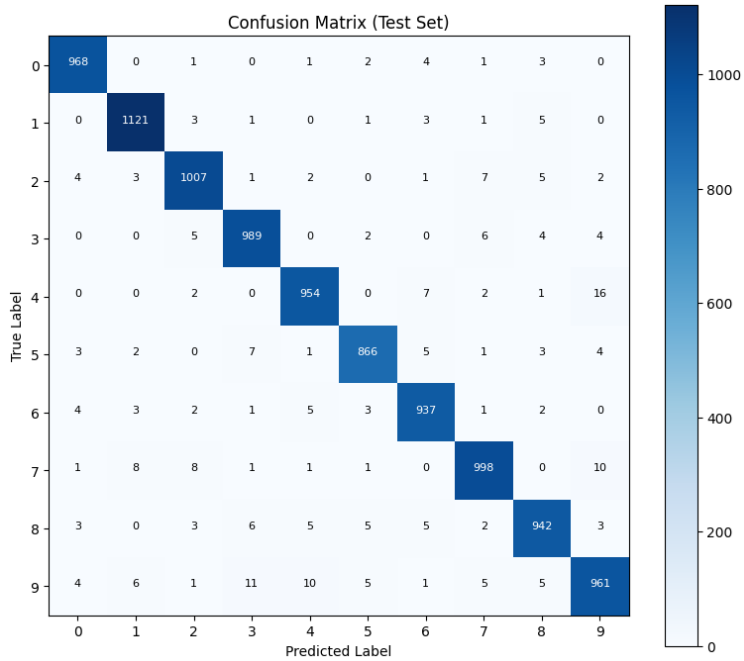

Figure 5: Confusion matrix on the test set for the 10-class MLP. Strong diagonal indicates high accuracy across all digit classes.

Training curves. Figure 6 shows the training and validation loss and validation accuracy over 67 epochs. The gap between training and validation loss increases in later epochs, indicating some overfitting, which the early stopping mechanism helps control.

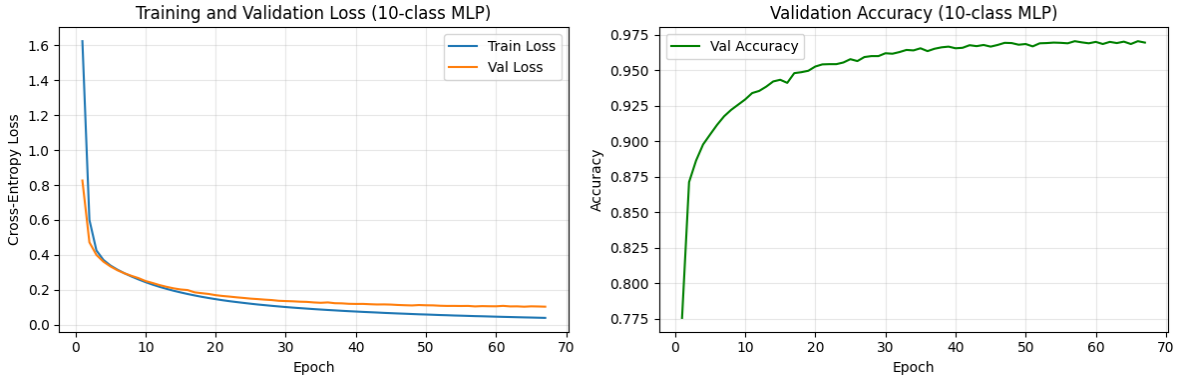

Figure 6: Training and validation loss (left) and validation accuracy (right) for the 10-class MLP. Early stopping triggered at epoch 67.

3.1 Python Code

The complete implementation is shown below. The code was executed on Google Colab with GPU acceleration (PyTorch 2.10.0, CUDA enabled).

```
transform = transforms.Compose([transforms.ToTensor()])
train_dataset = datasets.MNIST(root='./data', train=True,
                               download=True, transform=transform)
```

```python
test_dataset = datasets.MNIST(root='./data', train=False,
                                download=True, transform=transform)

X_train_full = train_dataset.data.float().view(-1, 28*28) / 255.0
y_train_full = train_dataset.targets.long()
X_test = test_dataset.data.float().view(-1, 28*28) / 255.0
y_test = test_dataset.targets.long()

n_total = len(X_train_full)
n_val = int(0.1 * n_total)
torch.manual_seed(42)
indices = torch.randperm(n_total)
X_train = X_train_full[indices[:n_total - n_val]]
y_train = y_train_full[indices[:n_total - n_val]]
X_val = X_train_full[indices[n_total - n_val:]]
y_val = y_train_full[indices[n_total - n_val:]]

batch_size = 64
train_loader = DataLoader(TensorDataset(X_train, y_train),
                          batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val),
                        batch_size=batch_size, shuffle=False)
test_loader = DataLoader(TensorDataset(X_test, y_test),
                         batch_size=batch_size, shuffle=False)
```

Listing 15: Data loading (full MNIST, all 10 classes).

```python
class MLP_Multiclass(nn.Module):
    """
    z_i = B * f(A * x_i + beta)
    p(Y=y|X) = softmax(W z_i + b), with w_9=0, b_9=0
    """
    def __init__(self, input_dim=784, d1=128, d2=64, num_classes=10):
        super(MLP_Multiclass, self).__init__()
        self.layer_A = nn.Linear(input_dim, d1)      # A, beta
        self.f = nn.ReLU()                            # f(.)
        self.layer_B = nn.Linear(d1, d2, bias=False)  # B
        # Only 9 output logits; class 9 fixed at zero
        self.output = nn.Linear(d2, num_classes - 1)

    def forward(self, x):
        hidden = self.f(self.layer_A(x))
        z = self.layer_B(hidden)
        logits_9 = self.output(z)
        # Concatenate zero logit for class 9
        zero_logit = torch.zeros(x.size(0), 1, device=x.device)
        logits_10 = torch.cat([logits_9, zero_logit], dim=1)
        return logits_10

model = MLP_Multiclass(input_dim=784, d1=128, d2=64).to(device)
```

Listing 16: MLP with softmax output and reference class constraint.

```python
criterion = nn.CrossEntropyLoss()
learning_rate = 0.01
optimizer = optim.SGD(model.parameters(), lr=learning_rate)
num_epochs = 100
patience = 10

```

```python
train_losses, val_losses, val_accuracies = [], [], []
best_val_acc, best_model_state, epochs_no_improve = 0.0, None, 0

for epoch in range(num_epochs):
    model.train()
    epoch_loss = 0.0
    for X_batch, y_batch in train_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        logits = model(X_batch)
        loss = criterion(logits, y_batch)
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item() * len(X_batch)
    train_losses.append(epoch_loss / len(X_train))

    model.eval()
    val_loss, correct = 0.0, 0
    with torch.no_grad():
        for X_batch, y_batch in val_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            val_loss += loss.item() * len(X_batch)
            preds = logits.argmax(dim=1)
            correct += (preds == y_batch).sum().item()
    val_losses.append(val_loss / len(X_val))
    val_accuracies.append(correct / len(X_val))

    if val_accuracies[-1] > best_val_acc:
        best_val_acc = val_accuracies[-1]
        best_model_state = {k: v.clone()
                            for k, v in model.state_dict().items()}
        epochs_no_improve = 0
    else:
        epochs_no_improve += 1
    if epochs_no_improve >= patience:
        break

model.load_state_dict(best_model_state)
```

Listing 17: Training loop with cross-entropy loss and early stopping.

```python
model.eval()
correct, total = 0, 0
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch, y_batch = X_batch.to(device), y_batch.to(device)
        logits = model(X_batch)
        preds = logits.argmax(dim=1)
        correct += (preds == y_batch).sum().item()
        total += len(y_batch)
test_accuracy = correct / total
print(f"TEST ACCURACY: {test_accuracy:.4f} ({correct}/{total})")
```

Listing 18: Test evaluation.

4 Question 4: Skip Connections and Layer Normalization

4.1 (a) Approach: Ablation Study

To study the impact of skip connections and layer normalization on classification performance, we conduct an ablation study over four model variants that differ only in whether these two mechanisms are enabled:

Table 9: Ablation study design.

Variant	Skip Connection	Layer Norm	Parameters
(A) Baseline	No	No	113,481
(B) + LayerNorm	No	Yes	113,993
(C) + Skip	Yes	No	113,481
(D) + Skip + LayerNorm	Yes	Yes	113,993

Architecture. To accommodate a skip connection, we extend the Q3 architecture by adding a third hidden layer that maps $\mathbb{R}^{64} \to \mathbb{R}^{64}$, yielding a dimension-matched residual block. The full forward pass is:

$$x_i \in \mathbb{R}^{784} \xrightarrow{A,\beta} \mathbb{R}^{128} \xrightarrow{\text{ReLU}} \mathbb{R}^{128} \xrightarrow{C} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{D,\delta} \mathbb{R}^{64} \xrightarrow{\text{ReLU}} \mathbb{R}^{64} \xrightarrow{W,b} \mathbb{R}^{10}.$$

Skip connection. The residual connection wraps around the third hidden layer. Let h_2 denote the output of the second hidden layer. The skip connection computes:

$$h_3 = f(Dh_2 + \delta) + h_2, \tag{7}$$

where $D \in \mathbb{R}^{64\times 64}$ and $\delta \in \mathbb{R}^{64}$. Because h_2 and $Dh_2+\delta$ have the same dimensionality, no projection is needed for the residual path. This allows gradients to flow directly through the identity branch during backpropagation, mitigating gradient degradation.

Layer normalization. When enabled, layer normalization is applied after each linear transform and before each activation function:

$$\text{LN}(h) = \gamma \odot \frac{h - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_{\text{LN}}, \tag{8}$$

where μ and σ^2 are the mean and variance computed across the feature dimension, and $\gamma, \beta_{\text{LN}} \in \mathbb{R}^d$ are learnable affine parameters. This adds $2 \times (128 + 64 + 64) = 512$ parameters to the model (the difference between 113,993 and 113,481). Layer normalization stabilizes the distribution of activations across layers, which can accelerate training and improve generalization.

All other settings. The output layer, loss function, identifiability constraint ($w_9 = \mathbf{0}$, $b_9 = 0$), dataset splits (54,000 / 6,000 / 10,000), and training hyperparameters are identical to Question 3: SGD with learning rate 0.01, batch size 64, maximum 100 epochs, and early stopping with patience 10. All four variants are trained with the same random seed (`torch.manual_seed(42)`) for a fair comparison.

4.2 (b) Results

Summary. Table 10 reports the results of the ablation study. The best-performing variant is **(D) + Skip + LayerNorm**, achieving a test accuracy of **97.94%** (9,794 / 10,000).

Table 10: Ablation study results. All models trained with identical hyperparameters and random seed.

Variant	Params	Val Acc	Test Acc	Epochs
(A) Baseline	113,481	96.88%	97.10%	51
(B) + LayerNorm	113,993	97.68%	97.81%	32
(C) + Skip	113,481	97.27%	97.21%	50
(D) + Skip + LayerNorm	113,993	97.67%	97.94%	35

Analysis.

- **Depth alone can hurt.** The Q4 baseline (A) achieves 97.10%, which is *lower* than Q3's 97.43% despite having more parameters (113,481 vs. 109,257) and an additional hidden layer. This is a textbook example of the *degradation problem*: naively stacking layers does not guarantee better performance, because the extra depth makes optimization harder. The added layer must learn to approximate an identity mapping to at least match the shallower network, which SGD does not achieve automatically. This observation directly motivates both skip connections (which provide an explicit identity shortcut) and layer normalization (which smooths the loss landscape).

- **Layer normalization is the dominant factor.** Comparing (A) vs. (B) and (C) vs. (D), adding layer normalization consistently improves test accuracy by $\sim$0.7 percentage points. It also dramatically accelerates convergence: models with LayerNorm converge in 32–35 epochs versus 50–51 without it. This is because normalizing activations reduces internal covariate shift, allowing higher effective learning rates and smoother optimization.

- **Skip connections provide a modest benefit.** Comparing (A) vs. (C), the skip connection alone improves test accuracy only slightly (97.10% $\rightarrow$ 97.21%). However, when combined with layer normalization in variant (D), skip connections contribute an additional 0.13 percentage points over (B). The relatively small gain is expected given the shallow depth of the network; skip connections become increasingly important in deeper architectures where gradient degradation is more severe.

- **Combined effect.** The best variant (D) achieves 97.94%, an improvement of 0.84 percentage points over the baseline (A) and 0.51 percentage points over Q3 (97.43%). The combination is complementary: layer normalization stabilizes training dynamics while the skip connection preserves representational capacity through the identity path. Notably, variant (D) not only recovers the accuracy lost by adding depth but surpasses Q3, confirming that skip connections and layer normalization are what make deeper networks viable.

Comparison with Q3. Table 11 compares the Q3 baseline with the best Q4 variant.

Table 11: Comparison of Q3 MLP and Q4 best variant (D).

Model	**Params**	**Val Acc**	**Test Acc**	**Epochs**
Q3: MLP (2 hidden layers)	109,257	97.03%	97.43%	67
Q4(D): MLP + Skip + LayerNorm (3 hidden layers)	113,993	97.67%	97.94%	35

The Q4 model improves test accuracy by 0.51 percentage points while converging nearly twice as fast (35 vs. 67 epochs), demonstrating that skip connections and layer normalization improve both final performance and training efficiency.

Training curves. Figure 7 compares the training loss, validation loss, and validation accuracy across all four variants. The LayerNorm variants (B and D) show faster initial convergence and lower final validation loss. The skip-only variant (C) tracks close to the baseline (A).

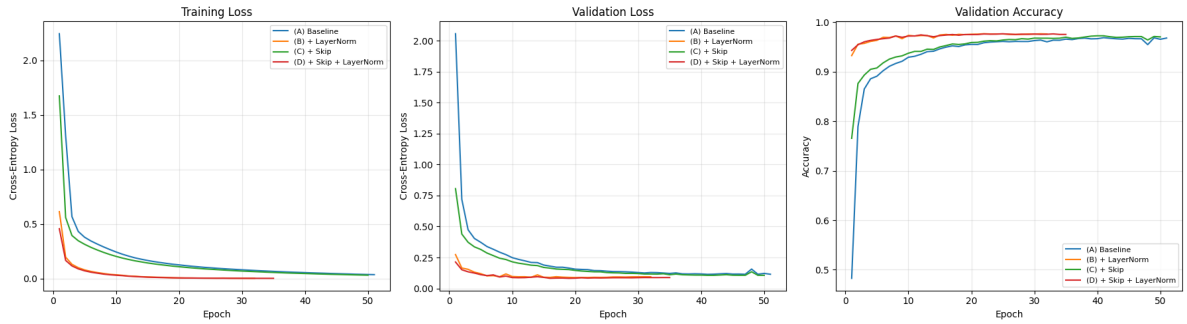

Figure 7: Training loss (left), validation loss (center), and validation accuracy (right) for all four ablation variants.

Test accuracy comparison. Figure 8 provides a bar chart comparing test accuracies across the four variants.

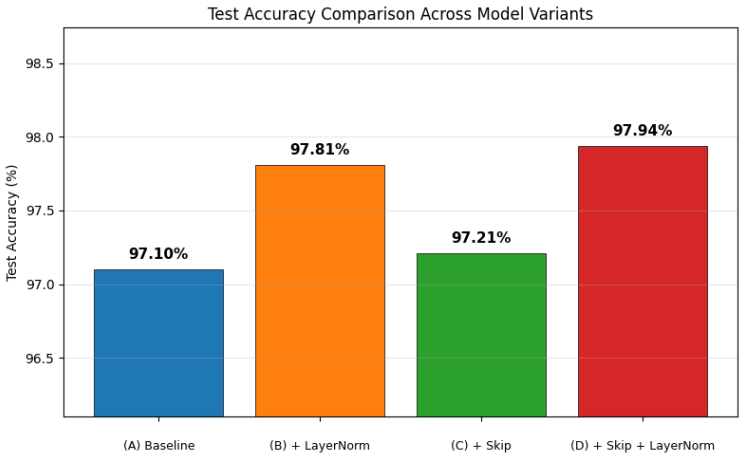

Figure 8: Test accuracy comparison across the four model variants.

Per-class accuracy (best variant). Table 12 reports per-class test accuracy for variant (D). All classes achieve above 96.9% accuracy. Compared to Q3, the most notable improvements are in

previously weaker classes: digit 9 improves from 95.24% to 96.93%, digit 8 from 96.71% to 97.13%, and digit 5 from 97.09% to 97.98%.

Table 12: Per-class test accuracy for variant (D) + Skip + LayerNorm.

Digit	**Correct**	**Total**	**Accuracy**
0	970	980	98.98%
1	1127	1135	99.30%
2	1007	1032	97.58%
3	994	1010	98.42%
4	961	982	97.86%
5	874	892	97.98%
6	935	958	97.60%
7	1002	1028	97.47%
8	946	974	97.13%
9	978	1009	96.93%

Confusion matrix. Figure 9 shows the confusion matrix for variant (D). The strong diagonal and low off-diagonal counts confirm consistently high classification across all digit classes.

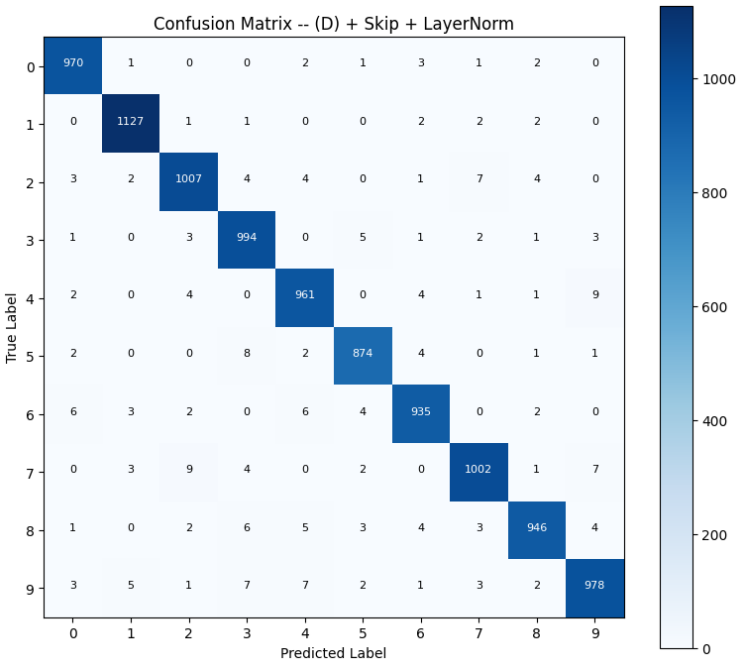

Figure 9: Confusion matrix on the test set for variant (D) + Skip + LayerNorm.

4.3 (c) Python Code

The complete implementation is shown below. The code was executed on Google Colab with GPU acceleration (PyTorch 2.10.0, CUDA enabled). The full notebook with running results is also available as `Q4_Skip_LayerNorm.ipynb`.

```python
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset
from torchvision import datasets, transforms
import matplotlib.pyplot as plt
import numpy as np

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
```

Listing 19: Imports and device setup.

```python
transform = transforms.Compose([transforms.ToTensor()])
train_dataset = datasets.MNIST(root='./data', train=True,
                                download=True, transform=transform)
test_dataset = datasets.MNIST(root='./data', train=False,
                               download=True, transform=transform)

X_train_full = train_dataset.data.float().view(-1, 28*28) / 255.0
y_train_full = train_dataset.targets.long()
X_test = test_dataset.data.float().view(-1, 28*28) / 255.0
y_test = test_dataset.targets.long()

n_total = len(X_train_full)
n_val = int(0.1 * n_total)
torch.manual_seed(42)
indices = torch.randperm(n_total)
X_train = X_train_full[indices[:n_total - n_val]]
y_train = y_train_full[indices[:n_total - n_val]]
X_val = X_train_full[indices[n_total - n_val:]]
y_val = y_train_full[indices[n_total - n_val:]]

batch_size = 64
train_loader = DataLoader(TensorDataset(X_train, y_train),
                          batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, y_val),
                        batch_size=batch_size, shuffle=False)
test_loader = DataLoader(TensorDataset(X_test, y_test),
                         batch_size=batch_size, shuffle=False)
```

Listing 20: Data loading (identical to Q3).

```python
class MLP_Ablation(nn.Module):
    """
    Two-hidden-layer MLP with configurable skip connection and layer norm.
    Architecture: 784 -> 128 (ReLU) -> 64 (ReLU) -> 64 (ReLU, skip+LN) -> 10
    Skip connection wraps around the 64->64 layer.
    Layer norm applied after linear, before activation.
    """
    def __init__(self, input_dim=784, d1=128, d2=64,
                 num_classes=10, use_skip=False, use_layernorm=False):
        super(MLP_Ablation, self).__init__()
        self.use_skip = use_skip
        self.use_layernorm = use_layernorm

        # Layer 1: 784 -> 128
        self.layer1 = nn.Linear(input_dim, d1)
        self.ln1 = nn.LayerNorm(d1) if use_layernorm else nn.Identity()
```

```python

        # Layer 2: 128 -> 64
        self.layer2 = nn.Linear(d1, d2)
        self.ln2 = nn.LayerNorm(d2) if use_layernorm else nn.Identity()

        # Layer 3 (residual block): 64 -> 64
        self.layer3 = nn.Linear(d2, d2)
        self.ln3 = nn.LayerNorm(d2) if use_layernorm else nn.Identity()

        self.relu = nn.ReLU()

        # Output: 9 logits (class 9 fixed at 0)
        self.output = nn.Linear(d2, num_classes - 1)

    def forward(self, x):
        # Layer 1
        h1 = self.relu(self.ln1(self.layer1(x)))
        # Layer 2
        h2 = self.relu(self.ln2(self.layer2(h1)))
        # Layer 3 with optional skip connection
        h3 = self.ln3(self.layer3(h2))
        if self.use_skip:
            h3 = h3 + h2  # Residual (dimensions match: 64 -> 64)
        h3 = self.relu(h3)
        # Output
        logits_9 = self.output(h3)
        zero_logit = torch.zeros(x.size(0), 1, device=x.device)
        return torch.cat([logits_9, zero_logit], dim=1)
```

Listing 21: MLP with configurable skip connection and layer normalization.

```python
def train_model(model, train_loader, val_loader, device,
                lr=0.01, max_epochs=100, patience=10):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(model.parameters(), lr=lr)

    train_losses, val_losses, val_accuracies = [], [], []
    best_val_acc, best_model_state, epochs_no_improve = 0.0, None, 0

    for epoch in range(max_epochs):
        model.train()
        epoch_loss = 0.0
        for X_batch, y_batch in train_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            optimizer.zero_grad()
            logits = model(X_batch)
            loss = criterion(logits, y_batch)
            loss.backward()
            optimizer.step()
            epoch_loss += loss.item() * len(X_batch)
        train_losses.append(epoch_loss / len(X_train))

        model.eval()
        val_loss, correct = 0.0, 0
        with torch.no_grad():
            for X_batch, y_batch in val_loader:
                X_batch, y_batch = X_batch.to(device), y_batch.to(device)
                logits = model(X_batch)
```

```
                loss = criterion(logits, y_batch)
                val_loss += loss.item() * len(X_batch)
                preds = logits.argmax(dim=1)
                correct += (preds == y_batch).sum().item()
        val_losses.append(val_loss / len(X_val))
        val_acc = correct / len(X_val)
        val_accuracies.append(val_acc)

        if val_acc > best_val_acc:
            best_val_acc = val_acc
            best_model_state = {k: v.clone()
                                for k, v in model.state_dict().items()}
            epochs_no_improve = 0
        else:
            epochs_no_improve += 1
        if epochs_no_improve >= patience:
            break

    model.load_state_dict(best_model_state)
    return model, {
        "train_losses": train_losses, "val_losses": val_losses,
        "val_accuracies": val_accuracies,
        "best_val_acc": best_val_acc, "epochs": len(train_losses),
    }

def evaluate_model(model, test_loader, device):
    model.eval()
    correct, total = 0, 0
    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            X_batch, y_batch = X_batch.to(device), y_batch.to(device)
            logits = model(X_batch)
            preds = logits.argmax(dim=1)
            correct += (preds == y_batch).sum().item()
            total += len(y_batch)
    return correct / total, correct, total
```

Listing 22: Training and evaluation functions.

```
variants = {
    "(A) Baseline":          {"use_skip": False, "use_layernorm": False},
    "(B) + LayerNorm":       {"use_skip": False, "use_layernorm": True},
    "(C) + Skip":            {"use_skip": True,  "use_layernorm": False},
    "(D) + Skip + LayerNorm":{"use_skip": True,  "use_layernorm": True},
}

results = {}
for name, kwargs in variants.items():
    torch.manual_seed(42)
    model = MLP_Ablation(**kwargs).to(device)
    model, history = train_model(model, train_loader, val_loader, device,
                                 lr=0.01, max_epochs=100, patience=10)
    test_acc, correct, total = evaluate_model(model, test_loader, device)
    results[name] = {
        "model": model, "history": history,
        "test_acc": test_acc, "correct": correct, "total": total,
        "n_params": sum(p.numel() for p in model.parameters()),
    }
    print(f"{name}: Epochs={history['epochs']}, "
```

```
          f"Val Acc={history['best_val_acc']:.4f}, "
          f"Test Acc={test_acc:.4f} ({correct}/{total})")
```

Listing 23: Run all four variants.

```
# Training curves (all 4 variants)
fig, axes = plt.subplots(1, 3, figsize=(18, 5))
colors = {"(A) Baseline": "tab:blue", "(B) + LayerNorm": "tab:orange",
          "(C) + Skip": "tab:green", "(D) + Skip + LayerNorm": "tab:red"}
for ax, key, ylabel in zip(axes, ['train_losses','val_losses','val_accuracies'],
                           ['Cross-Entropy Loss','Cross-Entropy Loss','Accuracy'
                              ]):
    for name, r in results.items():
        epochs = range(1, r['history']['epochs'] + 1)
        ax.plot(epochs, r['history'][key], label=name, color=colors[name])
    ax.set_xlabel('Epoch'); ax.set_ylabel(ylabel); ax.legend(fontsize=8)
    ax.grid(True, alpha=0.3)
axes[0].set_title('Training Loss')
axes[1].set_title('Validation Loss')
axes[2].set_title('Validation Accuracy')
plt.tight_layout()
plt.savefig('q4_training_comparison.png', dpi=150, bbox_inches='tight')
plt.show()

# Bar chart
fig, ax = plt.subplots(figsize=(8, 5))
names = list(results.keys())
test_accs = [results[n]['test_acc'] * 100 for n in names]
bars = ax.bar(range(len(names)), test_accs,
              color=[colors[n] for n in names], edgecolor='black')
for bar, acc in zip(bars, test_accs):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + 0.05,
            f'{acc:.2f}%', ha='center', va='bottom', fontweight='bold')
ax.set_xticks(range(len(names)))
ax.set_xticklabels(names, fontsize=9)
ax.set_ylabel('Test Accuracy (%)')
ax.set_title('Test Accuracy Comparison')
plt.tight_layout()
plt.savefig('q4_test_accuracy_bar.png', dpi=150, bbox_inches='tight')
plt.show()
```

Listing 24: Training curves comparison and bar chart.

```
from sklearn.metrics import confusion_matrix
import itertools

best_name = max(results, key=lambda n: results[n]['test_acc'])
best_model = results[best_name]['model']
best_model.eval()
all_preds, all_labels = [], []
with torch.no_grad():
    for X_batch, y_batch in test_loader:
        X_batch = X_batch.to(device)
        logits = best_model(X_batch)
        all_preds.append(logits.argmax(dim=1).cpu())
        all_labels.append(y_batch)
all_preds = torch.cat(all_preds)
all_labels = torch.cat(all_labels)

```

```python
for c in range(10):
    mask = (all_labels == c)
    acc = (all_preds[mask] == c).sum().item() / mask.sum().item()
    print(f"Class {c}: {acc:.4f}")

cm = confusion_matrix(all_labels.numpy(), all_preds.numpy())
fig, ax = plt.subplots(1, 1, figsize=(8, 7))
im = ax.imshow(cm, interpolation='nearest', cmap='Blues')
plt.colorbar(im, ax=ax)
ax.set_xticks(range(10)); ax.set_yticks(range(10))
for i, j in itertools.product(range(10), range(10)):
    ax.text(j, i, format(cm[i, j], 'd'), ha='center',
            color='white' if cm[i, j] > cm.max()/2 else 'black', fontsize=8)
ax.set_ylabel('True Label'); ax.set_xlabel('Predicted Label')
ax.set_title(f'Confusion Matrix -- {best_name}')
plt.tight_layout()
plt.savefig('q4_confusion_matrix_best.png', dpi=150, bbox_inches='tight')
plt.show()
```

Listing 25: Per-class accuracy and confusion matrix for best variant.